

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA0202

COURSE OUTCOME COVERED

CO2: Apply image processing and feature extraction techniques, including edge detection, motion estimation, and optical flow, for analyzing images.. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Total

Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hashini S

Question 1: Salt-and-Pepper Noise Removal Before Edge Detection

Problem Understanding

Given

Parameter	Value
Image Size	1024 × 1024 pixels
Image Type	Grayscale
Noise Type	Salt-and-Pepper (Impulse Noise)
Noise Density	0.05 (5%)
Next Processing Step	Edge Detection

Quick Observation

Instead of immediately choosing a filter, first understand the problem.

Salt-and-pepper noise changes only a few pixels into **0 (black)** or **255 (white)** randomly.

Since edge detection depends on sudden intensity changes, these noisy pixels may be wrongly identified as edges.

Therefore the preprocessing filter must satisfy two conditions:

- ✓ Remove isolated noisy pixels
- ✓ Preserve actual object boundaries

(a) Most Suitable Filtering Technique

Answer

The **Median Filter** is the most suitable preprocessing technique.

Why?

Unlike averaging filters, the median filter does **not calculate the average intensity**.

Instead, it

1. Collects neighboring pixel values.
2. Arranges them in ascending order.

3. Replaces the center pixel with the **middle value (median)**.

Because noisy pixels (0 or 255) are extreme values, they naturally move to the beginning or end of the sorted list and rarely become the median.

Hence,

- Random impulse noise disappears.
- True edges remain almost unchanged.

Mini Illustration

Suppose a 3×3 neighborhood is

52 53 54

55 255 56

54 55 57

Sorted values

52 53 54 54 55 55 56 57 255

Median

55

So

255 $\rightarrow$ 55

Noise removed without affecting nearby pixels.

Final Statement

Since impulse noise affects only isolated pixels, the median filter effectively suppresses noise while preserving important edge information, making it the preferred preprocessing method.

(b) Why Median Filter is Better than Averaging Filter?

Instead of memorizing differences, think from the pixel's perspective.

Imagine one noisy white pixel surrounded by normal gray pixels.

Averaging Filter

Example

50 51 52

50 255 49

51 52 50

Average

≈ 73

The noisy value influences every surrounding pixel.

Result

Edges become blurred.

Median Filter

Sorted

49 50 50 50 51 51 52 52 255

Median

51

Only the noisy pixel changes.

Neighbouring pixels remain untouched.

Comparison Table

Averaging Filter	Median Filter
------------------	---------------

Uses mean value	Uses middle value
Sensitive to outliers	Resistant to outliers
Blurs edges	Preserves edges
Poor for impulse noise	Excellent for impulse noise

Observation

Averaging tries to **mix** noise with good pixels.

Median simply **removes** the abnormal pixel.

(c) Probability that a Noisy Center Pixel is Corrected

Given

Noise Density

$$p = 0.05$$

Window

$$3 \times 3$$

Total pixels

$$9$$

Assumption

Neighbouring noises occur independently.

For the median filter to fail,

at least **5 pixels** inside the window must also be noisy.

Probability of failure

$$P_{failure} = \sum_{k=5}^9 \binom{9}{k} (0.05)^k (0.95)^{9-k}$$

Since

$$p = 5\%$$

this probability is extremely small.

Approximation

$$P(\text{failure}) \approx 0.00003$$

Therefore

$$P(\text{correct}) = 1 - P(\text{failure})$$

$$P(\text{correct}) \approx 0.99997$$

or

99.997%

Interpretation

Almost every noisy center pixel will be restored correctly because most neighboring pixels are still original image pixels.

(d) Estimate Total Number of Noisy Pixels

Image size

$$1024 \times 1024$$

=

$$1,048,576 \text{ pixels}$$

Noise density

$$5\%$$

Noisy pixels

$$= 0.05 \times 1,048,576$$

$$= 52,428.8$$

Approximate

52,429 noisy pixels

Observation

Even though only 5% pixels are corrupted,
more than **fifty-two thousand pixels** contain noise.

This is sufficient to create many false edges.

(e) Expected Noisy Pixels Remaining After Median Filtering

From part (c),

Probability that correction fails

$\approx$

0.00003

Remaining noisy pixels

$$52429 \times 0.00003$$

$\approx$

1.57

Approximate

Only about 2 noisy pixels remain.

Engineering Interpretation

Initially

52,429 noisy pixels

↓

Median filtering

↓

≈ 2 noisy pixels

Noise reduction efficiency

>99.99%

This explains why median filtering is widely used before edge detection.

(f) What if the Noise Were Gaussian Instead?

This changes the whole strategy.

Salt-and-pepper noise

Few pixels become 0 or 255.

Gaussian noise

Every pixel receives a small random intensity variation.

Example

Original

120

Gaussian Noise

118

122

117

121

Every pixel changes slightly.

Median filtering is no longer the best choice.

Instead,

✓ Gaussian Filter

or

✓ Wiener Filter

is preferred.

Comparison

Salt & Pepper	Gaussian Noise
Random black/white pixels	Small variation in every pixel
Median Filter	Gaussian Filter
Removes impulse noise	Reduces random fluctuations

Preserves edges	Smooths noise but slightly blurs edges
-----------------	--

Practical Workflow

Salt & Pepper Noise

|



Median Filter

|



Edge Detection

Gaussian Noise

|



Gaussian Filter

|



Edge Detection

Conclusion

The **Median Filter** is the most appropriate preprocessing technique for the given surveillance image because it removes impulse noise while preserving object boundaries. Out of approximately **52,429 noisy pixels**, nearly all are corrected after filtering, leaving only about **2 residual noisy pixels** under the stated assumptions. If the image were instead corrupted by Gaussian noise, a **Gaussian filter** would be more suitable, as it effectively suppresses distributed random intensity variations before edge detection.